Assignment 3 CO3

Name: Riyaz Kamal

Register Number: 192572049

Course: DSA0117A

GitHub submission link:

https://github.com/Riyazkamal78/DSA0117A_assignments/tree/main/Assignment3_CO3_192572049

1) Constructors and Constructor Overloading

Design and object state

Book stores title, author and copies privately. The default and title only constructors delegate to the full constructor, so the nonnegative copy count is checked in one place. The initializer list constructs every member directly. A negative count throws before a Book finishes construction. Since copies has no public setter, normal client operations cannot bypass the invariant. An empty record can later be assigned a newly validated Book.

Construction rules

The compiler implicitly declares a default constructor only when the class has no user declared constructor or constructor template. Declaring Book(int) removes that implicit default construction option, so Book records[2] fails unless a usable default constructor is supplied. Initializer lists directly initialize members, support const members and references, and avoid default construction followed by assignment for class type members. Actual initialization follows member declaration order, regardless of the order written in the list. The controlled reorder example declares first before second but lists second first; -Wreorder diagnoses it. Both are independent constants, so the example safely prints 1 2 rather than reading an uninitialized member.

Explicit conversion experiment

The title only constructor is explicit because conversion from an arbitrary string into a donated library record should be deliberate. Book b = std::string("Gift"); compiles in explicit_demo.cpp without the macro, but fails when built with -DEXPLICIT=explicit. Book b(std::string("Gift")); remains valid with explicit. The actual compiler errors and warning are included in the diagnostic image and full logs.

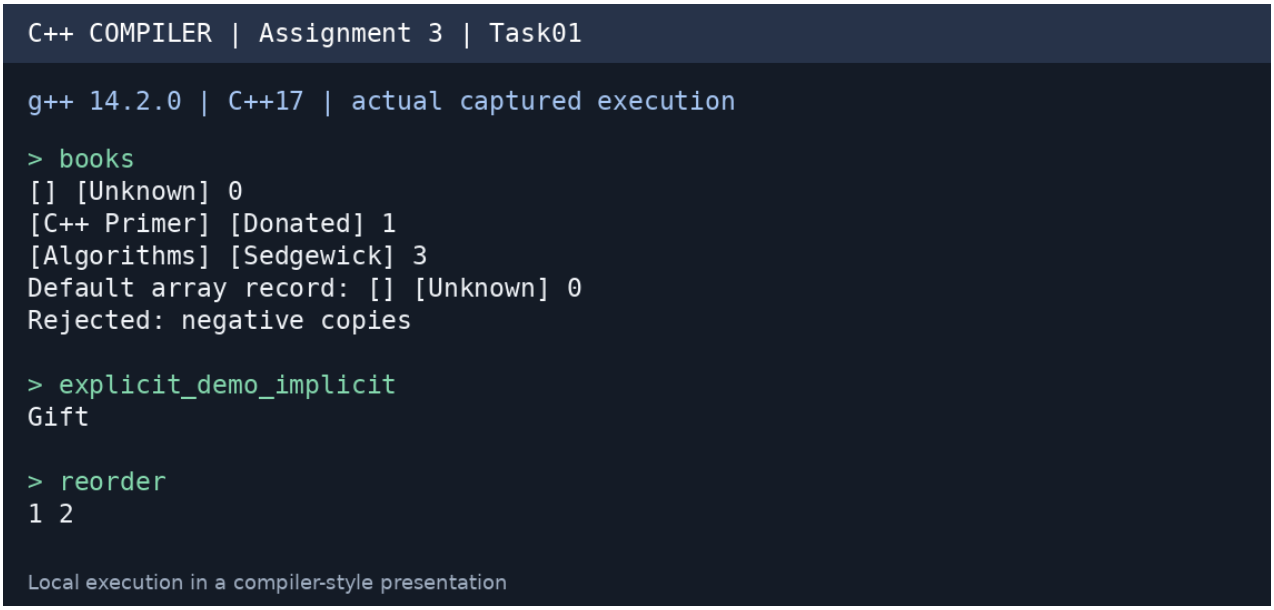
Alternative and complexity

A single constructor could be Book(std::string t="", int c=0, std::string a="Unknown") with the same validation in its initializer/body. Both that design and delegation validate the count once; overloading need not duplicate checks. Defaults are concise but make argument positions important and cannot independently encode a title only donation as one copy while an empty record has zero copies without additional policy. The fixed number of member operations is O(1) in the assignment model. More precisely, copying strings of total length L takes O(L) time and owned storage in either design; moving already constructed string arguments may avoid their payload copies.

State of three constructed objects

Object	Title	Author	Copies
empty	empty string	Unknown	0
donated	C++ Primer	Donated	1
full	Algorithms	Sedgewick	3

Execution evidence



C++ COMPILER | Assignment 3 | Task01

g++ 14.2.0 | C++17 | actual captured execution

```
> no_default
Task01/no_default.cpp: In function ‘int main()’:
Task01/no_default.cpp:5:28: error: no matching function for call to
‘Book::Book()’
    5 | int main() { Book records[2]; }
      |                            ^
Task01/no_default.cpp:3:5: note: candidate: ‘Book::Book(int)’
[excerpt; full transcript supplied in task folder]

> explicit_demo_explicit
Task01/explicit_demo.cpp: In function ‘int main()’:
Task01/explicit_demo.cpp:12:28: error: conversion from ‘std::string’ {aka
‘std::__cxx11::basic_string<char>’} to non-scalar type ‘Book’ requested
   12 | int main() { Book b = std::string("Gift"); b.print(); }
      |                            ^~~~~~~~~~~~~~

> reorder
Task01/reorder.cpp: In constructor ‘Example::Example()’:
Task01/reorder.cpp:3:16: warning: ‘Example::second’ will be initialized
after [-Wreorder]
    3 |     int first, second;
      |                ^~~~~~
Task01/reorder.cpp:3:9: warning:   ‘int Example::first’ [-Wreorder]
    3 |     int first, second;
      |         ^~~~~
Task01/reorder.cpp:4:5: warning:   when initialized here [-Wreorder]
[excerpt; full transcript supplied in task folder]
```

Local execution in a compiler-style presentation

1) Program sources

Each file is compiled separately. Diagnostic variants intentionally demonstrate a compiler error or invalid ownership; the corrected programs are identified in the answer and README.

books.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <utility>
class Book {
   std::string title, author;
   int copies;
public:
   Book() : Book("", "Unknown", 0) {}
   explicit Book(std::string t) : Book(std::move(t), "Donated", 1) {}
   Book(std::string t, std::string a, int c)
      : title(std::move(t)), author(std::move(a)), copies(c) {
      if (copies < 0) throw std::invalid_argument("negative copies");
   }
   void print() const {
      std::cout << '[' << title << "] [" << author << "] " << copies << '\n';
   }
};
int main() {
   Book empty, donated("C++ Primer"), full("Algorithms", "Sedgewick", 3);
   empty.print(); donated.print(); full.print();
   Book records[2];
   std::cout << "Default array record: "; records[0].print();
   try { Book invalid("Invalid", "Test", -1); }
   catch (const std::exception& e) { std::cout << "Rejected: " << e.what() << '\n'; }
}
```

explicit_demo.cpp

```
#include <iostream>
#include <string>
#ifndef EXPLICIT
#define EXPLICIT
#endif
class Book {
   std::string title;
public:
   EXPLICIT Book(std::string t) : title(t) {}
   void print() const { std::cout << title << '\n'; }
};
int main() { Book b = std::string("Gift"); b.print(); }
```

no_default.cpp

```
class Book {
public:
   Book(int) {}
};
int main() { Book records[2]; }
```

reorder.cpp

```
#include <iostream>
struct Example {
   int first, second;
   Example() : second(2), first(1) {}
};
int main() { Example e; std::cout << e.first << ' ' << e.second << '\n'; }
```

2) Shallow and Deep Copying of Prescriptions

Diagnose the shared ownership problem

The first shallow example has no copy constructor, so the implicitly generated constructor copies the pointer value. p and q then refer to the same medicine array: changing q changes p. Each destructor also believes it owns that same allocation. The first delete[] frees it; the second destructor accesses freed string array storage and attempts to release the allocation again. The actual sanitizer diagnosis is heap-use-after-free during destruction, rather than a guaranteed literal double-free message. Undefined behaviour has no portable output.

Deep copy and the Rule of Three

The completed Prescription allocates a fresh array in its copy constructor and copies each medicine. Assignment guards this==&other, builds and fills a temporary allocation first, then releases the old array and installs the new one. If string copying throws, the temporary allocation is deleted and the original destination remains unchanged. The destructor uses delete[]. The Rule of Three says that a class needing custom destruction, copying construction or copying assignment generally needs all three to express a consistent ownership policy. The exact completion retains the supplied constructor and accessors, so its caller must provide nonnegative counts and valid indices. The instrumented version validates these inputs and uses unique_ptr<string[]> internally to make cleanup automatic.

Copy situations and evidence

A copy constructor can initialize a new object from another object, initialize a by value parameter from an lvalue, or initialize a returned value when a copy is required. Copy assignment changes an already existing object and is different. C++17 guaranteed elision for appropriate prvalues and optional named return value optimization can remove copies; a move can be selected for eligible rvalues when available. The live lvalue pass by value test reports one copy, and the const reference test reports zero. Self assignment preserves Original, and a copied object independently contains Follow-up.

Completion and sanitizer validation

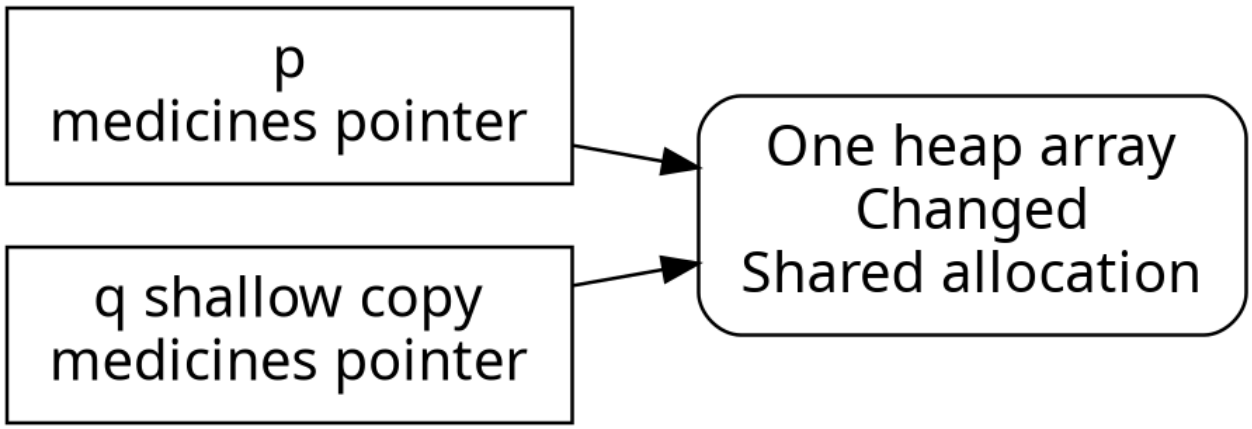
The completed program prints Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS, matching the saved prediction. Removing only its copy constructor produces the separate complete_no_copy.cpp variant; it fails under AddressSanitizer with heap-use-after-free. The shallow demonstration fails similarly. Both corrected implementations finish with exit status zero and no AddressSanitizer memory

access report when ASAN_OPTIONS=detect_leaks=0. Leak detection itself is unavailable here; an AddressSanitizer pass must not be read as proof of a leak-free process.

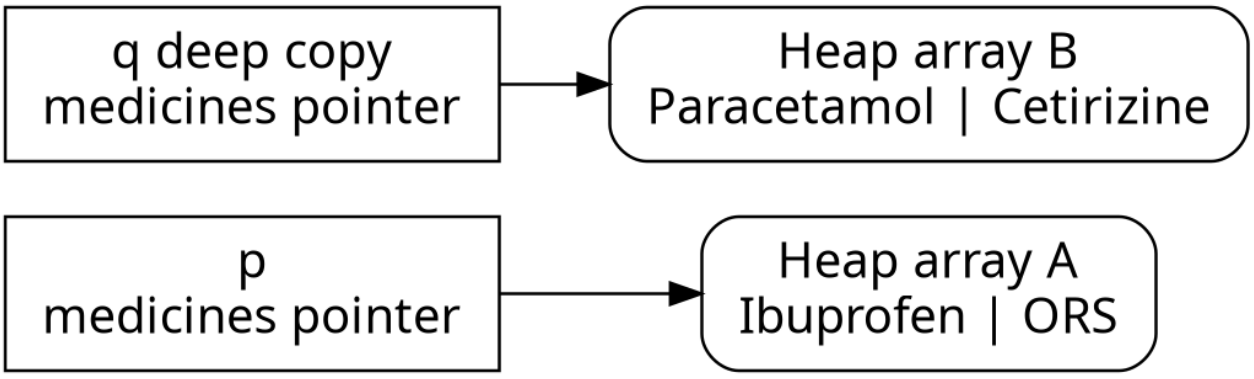
Measurements and alternative

The measured raw array copy test makes twenty copies for each of 10, 1000 and 100000 short medicine names. Allocation, copying and destruction are included; three -O2 trials and their medians are retained. Checksums are 200, 20000 and 2000000. The results support growth with the number of elements, but small timings contain timer and allocation noise. A Rule of Zero alternative is class Prescription { std::vector<std::string> medicines; }; with ordinary checked accessors and no custom copy or destructor. Vector copying already gives independent strings. Both correct designs use O(n) time and O(n) extra storage for n fixed length names, or O(n plus total characters) for arbitrary names. A shallow pointer copy uses O(1) time and O(1) extra storage but does not create independent ownership. Compiler generated copying is suitable for value type members such as string and vector; raw owning pointers require an explicit ownership design.

Memory diagrams



Shallow copy aliases one heap allocation



Deep copied arrays remain independently owned

Measured deep copy timings

Medicine count	20 copies median ms	Per copy median microseconds
10	0.001212	0.0606
1000	0.080961	4.0480
100000	10.108300	505.4150

Execution evidence

```
C++ COMPILER | Assignment 3 | Task02

g++ 14.2.0 | C++17 | actual captured execution

> complete_asan
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

> prescriptions
Copy constructor
Original | Follow-up
Copy constructor
Value size 2
Copies by value: 1
Reference size 2
Copies by const reference: 0
n=10 20 copies 0.001383 ms checksum 200
n=1000 20 copies 0.143995 ms checksum 20000
n=100000 20 copies 10.1737 ms checksum 2000000

> Sanitizer options
ASAN_OPTIONS=detect_leaks=0:halt_on_error=1
Corrected programs: exit 0; no invalid-access report.

Local execution in a compiler-style presentation
```

C++ COMPILER | Assignment 3 | Task02

g++ 14.2.0 | C++17 | actual captured execution

```
> shallow_asan
Original changed to: Changed
=================================================================
==59==ERROR: AddressSanitizer: heap-use-after-free on address
0x504000000010 at pc 0x55e1d9fee6cf bp 0x7ffd285e3c00 sp 0x7ffd285e3bf8
READ of size 8 at 0x504000000010 thread T0
    #0 0x55e1d9fee6ce in Prescription::~Prescription() Task02/shallow.cpp:7
    #1 0x55e1d9fee55c in main Task02/shallow.cpp:18
[excerpt; full transcript supplied in task folder]

> complete_no_copy_asan
=================================================================
==39==ERROR: AddressSanitizer: heap-use-after-free on address
0x507000000090 at pc 0x557b6d8716bd bp 0x7fff531f41d0 sp 0x7fff531f41c8
READ of size 8 at 0x507000000090 thread T0
    #0 0x557b6d8716bc in Prescription::~Prescription()
Task02/complete_no_copy.cpp:18
    #1 0x557b6d870bcc in main Task02/complete_no_copy.cpp:32
    #2 0x7ff75ed02ca7  (/lib/x86_64-linux-gnu/libc.so.6+0x29ca7) (BuildId:
5e5b51f4c39af9ac589408c1dc432ba390a273cc)
[excerpt; full transcript supplied in task folder]
```

Local execution in a compiler-style presentation

2) Program sources

Each file is compiled separately. Diagnostic variants intentionally demonstrate a compiler error or invalid ownership; the corrected programs are identified in the answer and README.

complete.cpp

```
#include <iostream>
#include <string>
class Prescription {
   int n;
   std::string* medicines;
public:
   explicit Prescription(int count) : n(count), medicines(new std::string[count]) {}
   Prescription(const Prescription& other)
      : n(other.n), medicines(new std::string[other.n]) {
      try {
         for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
      } catch (...) { delete[] medicines; throw; }
   }
   Prescription& operator=(const Prescription& other) {
      if (this == &other) return *this;
      std::string* fresh = new std::string[other.n];
      try {
         for (int i = 0; i < other.n; ++i) fresh[i] = other.medicines[i];
      } catch (...) { delete[] fresh; throw; }
      delete[] medicines;
      medicines = fresh; n = other.n;
      return *this;
   }
   ~Prescription() { delete[] medicines; }
   void set(int i, const std::string& m) { medicines[i] = m; }
   std::string get(int i) const { return medicines[i]; }
};
int main() {
```

```
    Prescription p(2);
    p.set(0, "Paracetamol"); p.set(1, "ORS");
    Prescription q = p;
    q.set(1, "Cetirizine");
    Prescription r(1); r = p;
    p.set(0, "Ibuprofen");
    std::cout << p.get(0) << ' ' << p.get(1) << " | "
              << q.get(0) << ' ' << q.get(1) << " | "
              << r.get(0) << ' ' << r.get(1) << '\n';
}
```

complete_no_copy.cpp

This controlled variant is complete.cpp with only the copy constructor removed. The full executable source and its failing sanitizer transcript are in Task02. All other supplied members remain present.

prescriptions.cpp

```
#include <chrono>
#include <iostream>
#include <stdexcept>
#include <string>
#include <memory>
#include <algorithm>
class Prescription {


    int n;
    std::unique_ptr<std::string[]> medicines;
public:
    static int copies;
    explicit Prescription(int count) : n(count), medicines(allocate(count)) {
        for (int i = 0; i < n; ++i) medicines[i] = "Medicine";
    }
    static std::unique_ptr<std::string[]> allocate(int n) {
        if (n < 0) throw std::invalid_argument("negative count");
        return std::unique_ptr<std::string[]>(new std::string[n]);
```

```
    }
    Prescription(const Prescription& p) : n(p.n), medicines(allocate(p.n)) {
        std::copy_n(p.medicines.get(), n, medicines.get());
        ++copies;
        if (n <= 2) std::cout << "Copy constructor\n";
    }
    Prescription& operator=(const Prescription& p) {
        if (this != &p) {
            auto fresh = allocate(p.n);
            std::copy_n(p.medicines.get(), p.n, fresh.get());
            medicines.swap(fresh); n = p.n;
        }
        return *this;
    }
    ~Prescription() = default;
    void set(int i, const std::string& s) {
        if (i < 0 || i >= n) throw std::out_of_range("medicine index");
        medicines[i] = s;
    }
    const std::string& get(int i) const {
        if (i < 0 || i >= n) throw std::out_of_range("medicine index");
        return medicines[i];
    }
    std::size_t size() const { return static_cast<std::size_t>(n); }

};
int Prescription::copies = 0;
void byValue(Prescription p) { std::cout << "Value size " << p.size() << '\n'; }
void byReference(const Prescription& p) { std::cout << "Reference size " << p.size() << '\n'; }
int main() {
    Prescription p(2); p.set(0, "Original");
    Prescription q = p; q.set(0, "Follow-up");
    p = p;
```

```
    std::cout << p.get(0) << " | " << q.get(0) << '\n';
    Prescription::copies = 0;
    byValue(p); std::cout << "Copies by value: " << Prescription::copies << '\n';
    Prescription::copies = 0;
    byReference(p); std::cout << "Copies by const reference: " << Prescription::copies << '\n';
    for (int n : {10, 1000, 100000}) {
        Prescription source(n);
        auto start = std::chrono::steady_clock::now();
        std::size_t checksum = 0;
        for (int i = 0; i < 20; ++i) {
            Prescription copy(source); checksum += copy.size();
        }
        double ms = std::chrono::duration<double, std::milli>(
            std::chrono::steady_clock::now() - start).count();
        std::cout << "n=" << n << " 20 copies " << ms << " ms checksum " << checksum << '\n';
    }
}
```

shallow.cpp

```
#include <iostream>
#include <string>
class Prescription {
    std::string* medicines;
public:
    Prescription() : medicines(new std::string[1]{"Original"}) {}
    ~Prescription() { delete[] medicines; }
    void set(const std::string& s) { medicines[0] = s; }
    const std::string& get() const { return medicines[0]; }
    // No copy constructor: compiler copies the pointer value.
};
int main() {
    Prescription p;
    Prescription q = p;
    q.set("Changed");
    std::cout << "Original changed to: " << p.get() << std::endl;
    // q frees the shared array first; p then uses the freed allocation.
}
```

3) Dynamic Constructors Destructors and Auditorium Seats

Allocation and class design

Auditorium allocates a char* row pointer array with new[], then one char seat array per row. Value initialization sets every seat to zero. book() rejects invalid or occupied seats, while cancel() rejects invalid or unoccupied seats. The count changes only on successful operations. Its destructor releases every row and then the pointer array and prints a closing summary. Copying is disabled to prevent accidental shared ownership. If row allocation fails partway through construction, the catch block releases the already initialized pointers; the object destructor cannot clean up a construction that never completed.

Lifetime order and prediction

A local object finishes construction before later statements run. Block objects are destroyed in reverse construction order when their block ends; automatic objects also unwind on exceptions. The heap object is destroyed when delete is executed, not when its pointer leaves scope. The run shows Stack created, Block created and destroyed, Heap created and deleted, then Zero and each temporary Measure event. At main exit Zero is destroyed before Stack. The Logger prediction is exactly +A +B -B copyAc use -Ac +P -P end -A followed by a space and no newline. b dies at its block end. use(a) copies the lvalue into its parameter, which dies when use returns. P dies at delete, and a dies after end is printed. Actual output matches.

Destructor rules and safety

A destructor takes no arguments and has no return type because it is invoked by the language for an existing object at lifetime end; in this C++17 program it cannot be overloaded. Forgetting delete for a new heap object leaks the object and its owned resources and omits its closing summary. new[] must be paired with delete[], and new with delete; mismatching them causes undefined behaviour. Releasing a row pointer array alone does not release its separate rows. RAII couples resource release to lifetime, making ordinary scope exit and exception paths safer. A raw heap owner can still be forgotten, so a production caller should use unique_ptr<Auditorium>.

Memory measurement and zero rows

Requested allocation payload is rows*sizeof(char*) + rows*seats*sizeof(char). With the measured eight byte pointers, 10 by 20 needs 280 bytes, 100 by 200 needs 20800, and 1000 by 2000 needs 2008000. These are the program calculated requested payload bytes, not allocator metadata, array cookies, process

RSS or object storage. For positive row counts there are rows+1 allocation requests. Zero rows keeps map=nullptr, requests no allocation, rejects booking and is safely destructible. Large dimensions can still exhaust memory; allocation failure is propagated after cleanup.

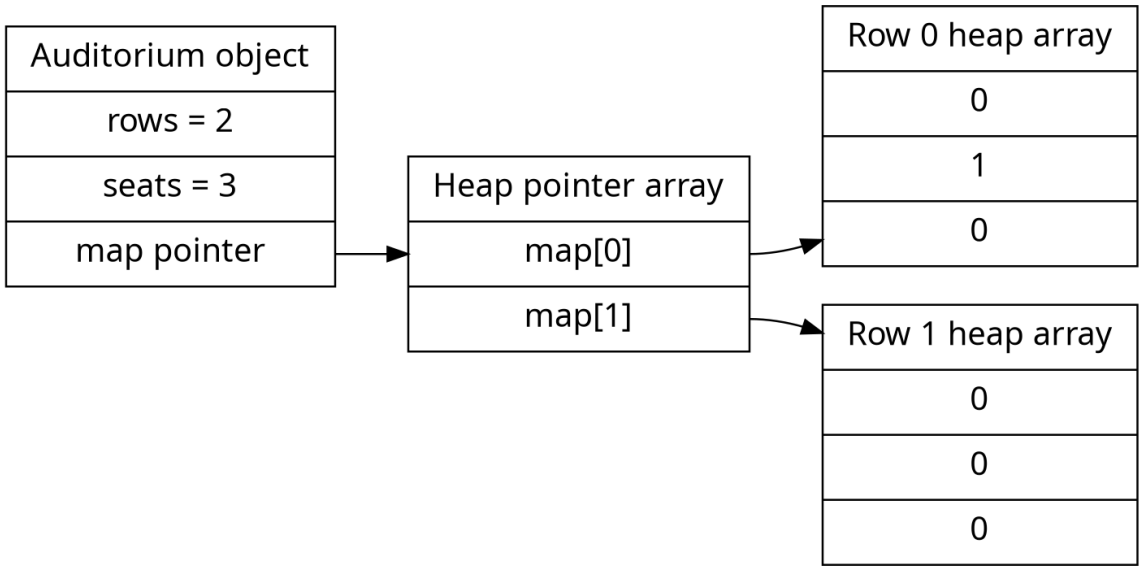
Missing delete experiment and environment limit

leak.cpp deliberately omits delete[] under -DLEAK, then runs the corresponding fixed build. Its reported outstanding payload is derived from the known 200 byte allocation and the presence of the delete, not a heap profiler measurement. The leak detector attempt was retained, but LeakSanitizer exits with a fatal error stating it does not work under ptrace. Consequently this environment cannot supply a valid detected leak report or a clean leak scan. AddressSanitizer with leak detection disabled can still execute both variants and check invalid memory accesses. The complete auditorium version also passes that memory access check. To finish the leak experiment independently, run the supplied missing-delete and fixed builds on a machine where ASAN_OPTIONS=detect_leaks=1 works.

Alternative layouts and complexity

A contiguous alternative allocates new char[rows*seats]{} and indexes map[r*seats+s]. It uses one allocation for a nonempty map and no row pointer array, often improving locality. Check the multiplication for overflow before allocating. A vector<vector<char>>(rows, vector<char>(seats)) gives value ownership and automatic cleanup but typically one outer allocation plus row allocations, with extra vector metadata; the standard does not promise an exact allocator request count. All three support O(1) indexed access. Zeroing construction costs O(rows*seats) time and payload storage; the pointer layout also costs O(rows) pointer storage and row deletion operations. A contiguous char array releases one block, while vector destruction releases its owned allocations automatically.

Seat map memory layout



Object pointer to a heap pointer array and separately allocated rows

Requested dynamic allocation payload

Rows by seats	Row pointer bytes	Seat bytes	Total bytes
10 by 20	80	200	280
100 by 200	800	20000	20800
1000 by 2000	8000	2000000	2008000

Execution evidence

```
C++ COMPILER | Assignment 3 | Task03

g++ 14.2.0 | C++17 | actual captured execution

> auditorium_asan
+Stack rows=2 seats=3 payload=22 bytes
+Block rows=1 seats=2 payload=10 bytes
Book true duplicate false cancel true
-Block closing booked=0
+Heap rows=2 seats=2 payload=20 bytes
-Heap closing booked=1
+Zero rows=0 seats=20 payload=0 bytes
Zero booking false
+Measure rows=10 seats=20 payload=280 bytes
-Measure closing booked=0
+Measure rows=100 seats=200 payload=20800 bytes
-Measure closing booked=0
+Measure rows=1000 seats=2000 payload=2008000 bytes
-Measure closing booked=0
-Zero closing booked=0
-Stack closing booked=0

> prediction
+A +B -B copyAc use -Ac +P -P end -A

Local execution in a compiler-style presentation
```

```
C++ COMPILER | Assignment 3 | Task03

g++ 14.2.0 | C++17 | actual captured execution

> leak_missing_delete
Allocated 200 bytes, first seat X
Outstanding payload: 200 bytes (delete omitted)

> leak_fixed
Allocated 200 bytes, first seat X
Outstanding payload: 0 bytes

> leak_leak_detection_attempt
==77==LeakSanitizer has encountered a fatal error.
==77==HINT: For debugging, try setting environment variable
LSAN_OPTIONS=verbosity=1:log_threads=1
==77==HINT: LeakSanitizer does not work under ptrace (strace, gdb, etc)

Local execution in a compiler-style presentation
```

3) Program sources

Each file is compiled separately. Diagnostic variants intentionally demonstrate a compiler error or invalid ownership; the corrected programs are identified in the answer and README.

auditorium.cpp

```
#include <iostream>
#include <stdexcept>
#include <string>
class Auditorium {
   std::string name;
   int rows, seats, booked = 0;
   char** map;
public:
   Auditorium(std::string n, int r, int s)
      : name(n), rows(r), seats(s), map(nullptr) {
      if (r < 0 || s < 0) throw std::invalid_argument("negative dimensions");
      if (rows > 0) map = new char*[rows]{};
      try {
         for (int i = 0; i < rows; ++i) map[i] = new char[seats]{};
      } catch (...) {
         for (int i = 0; i < rows; ++i) delete[] map[i];
         delete[] map; throw;
      }
      std::cout << '+' << name << " rows=" << rows << " seats=" << seats
            << " payload=" << bytes() << " bytes\n";
   }
   Auditorium(const Auditorium&) = delete;
   Auditorium& operator=(const Auditorium&) = delete;
   ~Auditorium() {
      for (int i = 0; i < rows; ++i) delete[] map[i];
      delete[] map;
      std::cout << '-' << name << " closing booked=" << booked << '\n';
   }
```

```
    bool book(int r, int s) {
        if (r < 0 || r >= rows || s < 0 || s >= seats || map[r][s]) return false;
        map[r][s] = 1; ++booked; return true;
    }
    bool cancel(int r, int s) {
        if (r < 0 || r >= rows || s < 0 || s >= seats || !map[r][s]) return false;
        map[r][s] = 0; --booked; return true;
    }
    std::size_t bytes() const {
        return static_cast<std::size_t>(rows) * sizeof(char*)
            + static_cast<std::size_t>(rows) * seats * sizeof(char);
    }
};
int main() {
    std::cout << std::boolalpha;
    Auditorium local("Stack", 2, 3);
    {
        Auditorium block("Block", 1, 2);
        std::cout << "Book " << block.book(0, 1) << " duplicate " << block.book(0, 1)
                  << " cancel " << block.cancel(0, 1) << '\n';
    }
    Auditorium* heap = new Auditorium("Heap", 2, 2);
    heap->book(1, 1); delete heap;
    Auditorium zero("Zero", 0, 20);
    std::cout << "Zero booking " << zero.book(0, 0) << '\n';
    for (int r : {10, 100, 1000}) {
        Auditorium measurement("Measure", r, 2*r);
    }
}
```

leak.cpp

```
#include <iostream>
#if defined(__GNUC__)
__attribute__((noinline))
#endif
void forget() {
    char* seatMap = new char[200];
    seatMap[0] = 'X';
    std::cout << "Allocated 200 bytes, first seat " << seatMap[0] << '\n';
#ifndef LEAK
    delete[] seatMap;
    std::cout << "Outstanding payload: 0 bytes\n";
#else
    std::cout << "Outstanding payload: 200 bytes (delete omitted)\n";
#endif
}
int main() { forget(); }
```

prediction.cpp

```
#include <iostream>
#include <string>
struct Logger {
    std::string n;
    Logger(const std::string& s) : n(s) { std::cout << '+' << n << ' '; }
    Logger(const Logger& o) : n(o.n + "c") { std::cout << "copy" << n << ' '; }
    ~Logger() { std::cout << '-' << n << ' '; }
};
void use(Logger) { std::cout << "use "; }
int main() {
    Logger a("A");
    { Logger b("B"); }
    use(a);
    Logger* p = new Logger("P"); delete p;
    std::cout << "end ";
}
```

4) Money Arithmetic and Stream Operators

Representation and operator design

Money stores the total number of paise in a long long, so carrying and borrowing follow integer arithmetic. The full implementation checks addition, subtraction and rupee conversion for overflow. operator+= is a member because it mutates the left operand and returns *this by reference, supporting chained updates. operator+ creates a separate result so a+b leaves both operands unchanged. The required completion uses a member + and ==; the full design uses symmetric nonmember friends for +, -, == and <, allowing the same conversions on either operand. A negative amount is allowed as a credit or debt and prints as Rs -0.25.

Streams and function call translation

For the member completion, a+b becomes a.operator+(b). For the nonmember design it becomes operator+(a,b), found through normal lookup and argument dependent lookup. The conventional out<<money operator is nonmember because its left operand is an ostream; adding a Money member would instead require Money on the left. The standard stream class cannot be extended with our own member function. Friendship permits reading the private paise without exposing mutable storage. operator>> parses decimal digits without using double, accepts a leading sign and at most two fractional digits, and sets failbit on malformed or overflowing input while leaving the target unchanged.

Correctness and readability

The completion prints Rs 200.00 and true. The full test verifies 0.75+0.50=1.25, 0.50-0.75=-0.25, comparisons, chained bill updates and signed input. The operator expression bill = price + charge; bill += adjustment is shorter and follows normal arithmetic notation. The named equivalent is bill = add(price,charge); addAssign(bill,adjustment);. Both must retain the same units and rounding policy. The supplied completion is limited to its nonnegative examples; the separate full class handles negative output and integer overflow.

Integer and double comparison

A million additions of one paise produce exactly Rs 10000.00 in the integer class. Adding 0.01 as a double gives 10000.000000171856 here, a rupee error of about 1.72e-7. The discrepancy comes from binary floating point representation and accumulated rounding, not from the overload mechanism. Fixed paise is suitable when all values have two decimals; interest, tax or exchange calculations may still need a defined rounding rule before converting to paise.

Engineering decision and alternatives

Adding or subtracting Money and comparing amounts have clear meanings. Multiplying Money by Money produces currency squared, so this class does not provide it. Multiplication by a dimensionless quantity and division by a quantity could be added with overflow and rounding policies; Money/Money could produce a ratio. The nonexplicit rupees constructor and nonmember + allow both 100+m and m+100. A member + only allows the latter through conversion of its parameter. Named add(a,b) and equals(a,b) can express either order explicitly. Every fixed width arithmetic or comparison operation uses O(1) time and space regardless of member or friend syntax. Parsing and printing depend on the number of digits, bounded for long long. The input parser deliberately rejects magnitudes above LLONG_MAX, including the asymmetric LLONG_MIN magnitude.

Execution evidence

```
C++ COMPILER | Assignment 3 | Task04

g++ 14.2.0 | C++17 | actual captured execution

> complete
Rs 200.00
true

> money
Carry: Rs 1.25
Negative: Rs -0.25
Bill: Rs 200.00
Mixed: Rs 100.75 | Rs 100.75
Compare: false true
Read: Rs 12.34
Read: Rs -0.25
Invalid rejected: true
Integer million: Rs 10000.00
Double million: 10000.000000171856 error 1.7185629985760897e-07

Local execution in a compiler-style presentation
```

4) Program sources

Each file is compiled separately. Diagnostic variants intentionally demonstrate a compiler error or invalid ownership; the corrected programs are identified in the answer and README.

complete.cpp

```
#include <iomanip>
#include <iostream>
class Money {
   long long paise;
public:
   Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}
   Money operator+(const Money& b) const {
      Money result; result.paise = paise + b.paise; return result;
   }
   bool operator==(const Money& b) const { return paise == b.paise; }
   friend std::ostream& operator<<(std::ostream& out, const Money& m) {
      return out << "Rs " << m.paise/100 << '.'
                 << std::setfill('0') << std::setw(2) << m.paise%100 << std::setfill(' ');
   }
};
int main() {
   Money a(120, 75), b(79, 25);
   Money c = a + b;
   std::cout << c << '\n' << std::boolalpha << (c == Money(200)) << '\n';
}
```

money.cpp

```
#include <cmath>
#include <iomanip>
#include <iostream>
#include <limits>
#include <sstream>
#include <stdexcept>
```

```
#include <string>
class Money {
    long long paise = 0;
    static long long sum(long long a, long long b) {
        if ((b > 0 && a > std::numeric_limits<long long>::max()-b) ||
            (b < 0 && a < std::numeric_limits<long long>::min()-b))
            throw std::overflow_error("money overflow");
        return a+b;
    }
public:
    Money(long long rupees = 0) {
        if (rupees > std::numeric_limits<long long>::max()/100 ||
            rupees < std::numeric_limits<long long>::min()/100)
            throw std::overflow_error("rupees overflow");
        paise = rupees*100;
    }
    static Money fromPaise(long long p) { Money m; m.paise=p; return m; }
    Money& operator+=(const Money& b) { paise=sum(paise,b.paise); return *this; }
    friend Money operator+(Money a, const Money& b) { return a+=b; }
    friend Money operator-(const Money& a, const Money& b) {
        long long x=a.paise, y=b.paise;
        if ((y>0 && x<std::numeric_limits<long long>::min()+y) ||
            (y<0 && x>std::numeric_limits<long long>::max()+y))
            throw std::overflow_error("money overflow");
        return fromPaise(x-y);
    }
    friend bool operator==(const Money& a,const Money& b) { return a.paise==b.paise; }
    friend bool operator<(const Money& a,const Money& b) { return a.paise<b.paise; }
    friend std::ostream& operator<<(std::ostream& out,const Money& m) {
        unsigned long long mag=m.paise<0 ? 0ULL-static_cast<unsigned long long>(m.paise)
                                         : static_cast<unsigned long long>(m.paise);
        char fill=out.fill();
        out << "Rs " << (m.paise<0?"-":"") << mag/100 << '.'
```

```
        << std::setfill('0') << std::setw(2) << mag%100;
      out.fill(fill); return out;
    }
    friend std::istream& operator>>(std::istream& in,Money& m) {
      std::string token; if (!(in>>token)) return in;
      try {
        bool negative=token[0]=='-';
        std::size_t begin=(negative || token[0]=='+')?1:0;
        std::size_t dot=token.find('.',begin);
        std::string whole=token.substr(begin,dot==std::string::npos?dot:dot-begin);
        std::string fraction=dot==std::string::npos?"":token.substr(dot+1);
        auto digits=[](const std::string& s) {
          return s.find_first_not_of("0123456789")==std::string::npos;
        };
        if (whole.empty() || !digits(whole) || !digits(fraction) || fraction.size()>2)
          throw std::invalid_argument("amount");
        while (fraction.size()<2) fraction+='0';
        unsigned long long w=std::stoull(whole), f=std::stoull(fraction);
        unsigned long long max=static_cast<unsigned long long>(std::numeric_limits<long
long>::max());
        if (w>max/100 || w*100>max-f) throw std::out_of_range("amount");
        long long p=static_cast<long long>(w*100+f);
        m=fromPaise(negative?-p:p);
      } catch (...) { in.setstate(std::ios::failbit); }
      return in;
    }
};
int main() {
    Money a=Money::fromPaise(75), b=Money::fromPaise(50);
    std::cout << "Carry: " << a+b << "\nNegative: " << b-a << '\n';
    Money bill=Money(120)+Money::fromPaise(7975); bill+=Money::fromPaise(25);
    std::cout << "Bill: " << bill << "\nMixed: " << 100+a << " | " << a+100 << '\n';
    std::cout << std::boolalpha << "Compare: " << (a==b) << ' ' << (b<a) << '\n';
```

```
    std::istringstream input("12.34 -0.25 bad"); Money value;
    while (input>>value) std::cout << "Read: " << value << '\n';
    std::cout << "Invalid rejected: " << input.fail() << '\n';
    Money exact; double approximate=0;
    for (int i=0;i<1000000;++i) { exact+=Money::fromPaise(1); approximate+=0.01; }
    std::cout << "Integer million: " << exact << '\n' << std::setprecision(17)
              << "Double million: " << approximate << " error " << approximate-10000.0 << '\n';
}
```

5) Increment Comparison and Playlist Access

Playback design and predictions

Time stores elapsed seconds and formats hours, minutes and seconds, rather than storing fields that need separate carry logic. Prefix is Time& operator++(); postfix is Time operator++(int), where the unused int parameter distinguishes its signature. Postfix snapshots the old state, advances the object and returns the snapshot. From 3599 seconds, a=t++ gives a=0:59:59 and t=1:00:00; b=++t then gives b=t=1:00:01. The separate Counter prediction is exactly 5 7 7 followed by a newline: a gets the old 5, while b and c see the new 7. Both predictions match execution.

Const access and boundary handling

Playlist has a mutable operator[] returning string& and a const overload returning const string&. This permits editing a mutable playlist and reading a const view without granting write access through it. Both validate a signed index before converting it to size_t and throw out_of_range for -1 or size. Merely printing an error then returning an invalid reference would be unsafe; throwing gives the caller a clear failure path. The simulation catches those errors, edits a track, and uses a const view to print it. It also verifies minute rollover at 59 seconds and hour rollover after 59:59.

Measured increment and access costs

benchmark.cpp performs ten million prefix and postfix increments at -O0 and -O2. Each loop uses a volatile observed scalar to keep the updates observable; both finish at 10000000. The returned postfix snapshot is unused, so optimization is allowed to remove its copy. Three trials and median times are reported below. Postfix semantically returns the old value as an independent value; for this class its implementation makes a copy, but a measurable slowdown is not guaranteed. Prefix avoids requesting that old value and is a good default for expensive class types when the previous state is not needed.

Alternative and complexity

An at(i) member can use the same bounds check and throw, making checked access explicit in its name. An unchecked operator[] simply returns tracks[i]; invalid use then has undefined behaviour, like a built in array. Successful checked and unchecked accesses take O(1) time and O(1) space, with a constant bounds test added in the checked form; exception handling costs are outside the successful access timing. The access benchmark reads ten million valid indices from a small integer playlist surrogate with checked and unchecked functions, the same varying indices and checksum 75000000. It isolates access overhead from string copying but is not a full music player benchmark. The timing includes index

generation and volatile reads; the optimizer may simplify checks when it can prove bounds. Time increment is O(1); the supplied elapsed time class assumes increments remain below LLONG_MAX and would need an overflow policy for unbounded use.

Median time for ten million operations

Build	Prefix ms	Postfix ms	Checked access ms	Unchecked access ms
O0	14.16490	22.58880	19.86330	14.27460
O2	2.54695	2.70046	7.26890	5.92365

The postfix median is higher in both measured builds, but the optimized difference is small. The checked surrogate also takes longer in these trials. These observations describe this workload and compiler; they do not establish a fixed cost for every overloaded operator or bounds check.

Execution evidence

```
C++ COMPILER | Assignment 3 | Task05

g++ 14.2.0 | C++17 | actual captured execution

> playlist
a=0:59:59 b=1:00:01 t=1:00:01
Minute rollover 0:01:00
Comparisons true true
Playing Edited song at 1:00:01
Rejected -1: track index
Rejected 2: track index

> prediction
5 7 7

Local execution in a compiler-style presentation
```

C++ COMPILER | Assignment 3 | Task05

g++ 14.2.0 | C++17 | actual captured execution

```
> benchmark_O0
prefix: 14.1649 ms checksum 10000000
postfix: 21.8959 ms checksum 10000000
checked access: 18.7653 ms checksum 75000000
unchecked access: 13.6894 ms checksum 75000000

> benchmark_O2
prefix: 2.41557 ms checksum 10000000
postfix: 3.19842 ms checksum 10000000
checked access: 7.33709 ms checksum 75000000
unchecked access: 5.92365 ms checksum 75000000
```

Local execution in a compiler-style presentation

5) Program sources

Each file is compiled separately. Diagnostic variants intentionally demonstrate a compiler error or invalid ownership; the corrected programs are identified in the answer and README.

benchmark.cpp

```
#include <chrono>
#include <iostream>
#include <stdexcept>
struct Time {
    long long seconds=0;
    Time& operator++() { ++seconds; return *this; }
    Time operator++(int) { Time old=*this; ++*this; return old; }
};
template<class F> void timed(const char* label,F f) {
    auto start=std::chrono::steady_clock::now();
    long long checksum=f();
    double ms=std::chrono::duration<double,std::milli>(
        std::chrono::steady_clock::now()-start).count();
    std::cout << label << ": " << ms << " ms checksum " << checksum << '\n';
}
struct Playlist {
    int values[16]{};
    int checked(int i) const {
        if (i<0 || i>=16) throw std::out_of_range("index");
        return values[i];
    }
    int unchecked(int i) const { return values[i]; }
};
int main() {
    constexpr int N=10000000;
    // Volatile object access prevents replacing all increments with one addition.
    volatile long long observed=0;
    timed("prefix",[&] {
```

```
    Time t;
    for (int i=0;i<N;++i) { t.seconds=observed; ++t; observed=t.seconds; }
    return t.seconds;
  });
  observed=0;
  timed("postfix",[&] {
    Time t;
    for (int i=0;i<N;++i) { t.seconds=observed; t++; observed=t.seconds; }
    return t.seconds;
  });
  Playlist p; for(int i=0;i<16;++i) p.values[i]=i;
  volatile int input=3;
  timed("checked access",[&] {
    long long sum=0;
    for(int i=0;i<N;++i) sum+=p.checked((input+i)%16);
    return sum;
  });
  timed("unchecked access",[&] {
    long long sum=0;
    for(int i=0;i<N;++i) sum+=p.unchecked((input+i)%16);
    return sum;
  });
}
```

playlist.cpp

```
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
class Time {
  long long seconds;
public:
  explicit Time(long long s=0) : seconds(s) {
```

```
        if (s<0) throw std::invalid_argument("negative time");
    }
    Time& operator++() { ++seconds; return *this; }
    Time operator++(int) { Time old=*this; ++*this; return old; }
    bool operator==(const Time& b) const { return seconds==b.seconds; }
    bool operator<(const Time& b) const { return seconds<b.seconds; }
    friend std::ostream& operator<<(std::ostream& out,const Time& t) {
        char fill=out.fill();
        out << t.seconds/3600 << ':' << std::setfill('0') << std::setw(2)
            << t.seconds/60%60 << ':' << std::setw(2) << t.seconds%60;
        out.fill(fill); return out;
    }
};
class Playlist {
    std::vector<std::string> tracks;
public:
    Playlist() : tracks{"Intro","Song"} {}
    std::size_t size() const { return tracks.size(); }
    std::string& operator[](int i) {
        if (i<0 || static_cast<std::size_t>(i)>=size()) throw std::out_of_range("track index");
        return tracks[i];
    }
    const std::string& operator[](int i) const {
        if (i<0 || static_cast<std::size_t>(i)>=size()) throw std::out_of_range("track index");
        return tracks[i];
    }
};
int main() {
    Time t(3599); Time a=t++; Time b=++t;
    std::cout << "a=" << a << " b=" << b << " t=" << t << '\n';
    Time minute(59); std::cout << "Minute rollover " << ++minute << '\n';
    std::cout << std::boolalpha << "Comparisons " << (b==t) << ' ' << (a<t) << '\n';
    Playlist list; list[1]="Edited song"; const Playlist& view=list;
```

```
    std::cout << "Playing " << view[1] << " at " << t << '\n';
    for (int i : {-1,static_cast<int>(list.size())}) {
        try { std::cout << list[i] << '\n'; }
        catch (const std::out_of_range& e) { std::cout << "Rejected " << i << ": " << e.what() << '\n'; }
    }
}
```

prediction.cpp

```
#include <iostream>
struct Counter {
    int v;
    Counter& operator++() { ++v; return *this; }
    Counter operator++(int) { Counter old=*this; ++v; return old; }
};
int main() {
    Counter c{5}; Counter a=c++; Counter b=++c;
    std::cout << a.v << ' ' << b.v << ' ' << c.v << '\n';
}
```

6) Text Ownership and Operator Overloading Rules

Ownership implementation and tests

Text owns a null terminated dynamic character array and its length. The constructor allocates space including the terminator; the copy constructor allocates independent storage, and assignment first allocates and fills replacement storage. The explicit self assignment guard avoids unnecessary work. delete[] in the destructor releases the buffer. Concatenation allocates a result of the combined length, copies both character sequences, and returns it. Equality compares lengths and contents, != negates ==, and << prints the stored sequence. Empty strings, a 100000 character message and self assignment all pass. The long copied value retains length 100000 after its original is emptied.

Rules and friendship

The requested operators ::, ., .*, ?: and sizeof cannot be overloaded: C++ reserves their built in language meanings. A programmer cannot invent an operator token, change precedence, associativity or operand count, or redefine an operation solely on built in types. At least one operand must have a class or enumeration type. In this C++17 design operator= must be a nonstatic member, so it operates on the identity and existing storage of its left object. operator+ and comparisons are symmetric free friends here; << needs a stream on the left. A free operator should be a friend only when it needs private access; otherwise public const accessors may suffice.

Copy and sanitizer evidence

With the ordinary optimized build, evaluating a+b+c reports zero copy constructor calls: the two named result objects benefit from return value optimization in this compiler. With -O0 -fno-elide-constructors it reports two copies, one for each returned named local, because Text has no move constructor. This is implementation evidence, not a guaranteed copy count for every compiler. The final C++17 prvalue initializations can still be subject to mandatory elision. The address sanitized version completes with no invalid-access report and exit status zero; leak detection is disabled because the environment cannot run LeakSanitizer.

Benchmark and evaluation

The -O2 benchmark compares one hundred thousand independent concatenations of five plus four characters against std::string, with matched checksums of 900000 and three trial medians. It includes operand construction, allocation and destruction. It does not measure repeatedly appending to one ever growing string, which would be a different workload. Small string optimization can reduce std::string

allocations, but its details are implementation dependent. These measured results are shown below. Writing Text demonstrates ownership, exception safe assignment, buffer sizes, null terminators and return value behaviour. A production SMS service should normally prefer std::string; a custom type needs a concrete requirement such as specialized allocation or representation, together with substantially more validation. Remaining limits include nonnull C string inputs, unchecked combined length overflow, byte based rather than Unicode aware semantics, and no move operations.

Alternative and complexity

With std::string, s=a+b+c and s=s already have safe value semantics without custom special members. If Text supplied +=, a by value alternative could be Text operator+(Text a,const Text& b) { a+=b; return a; }. The first lvalue operand is copied, += may allocate and copy again, and returning the parameter is not NRVO eligible; without a move constructor a further copy can occur. A move enabled or capacity reserving design can reduce that overhead. For the current const reference implementation, character writes excluding terminators are n1+n2 for the first result and n1+n2+n3 for the second, totaling 2*n1+2*n2+n3 when return copies are elided. Live intermediate storage peaks at O(n1+n2+n3) beyond the original operands, since the intermediate and final result can coexist. Optional return copies add linear work and temporary storage without changing the asymptotic O(n1+n2+n3) time for exactly three operands. A naive repeated append sequence over many pieces can become quadratic; reserving final capacity and appending is preferable when total length is known.

Median time for one hundred thousand concatenations

Implementation	Median ms	Checksum
Text	1.843320	900000
std::string	0.982729	900000

std::string is faster for this optimized short-string workload. That result, the missing move operations and the extra ownership code support choosing the standard type for deployment.

Execution evidence

```
C++ COMPILER | Assignment 3 | Task06

g++ 14.2.0 | C++17 | actual captured execution

> text_O2
Hello SMS copies=0
Self assignment true empty true different true
Long independent copy size 100000
Text 100000 concatenations: 1.84762 ms checksum 900000
std::string 100000 concatenations: 0.982729 ms checksum 900000

> text_no_elide
Hello SMS copies=2
Self assignment true empty true different true
Long independent copy size 100000
Text 100000 concatenations: 5.88042 ms checksum 900000
std::string 100000 concatenations: 7.05118 ms checksum 900000

> AddressSanitizer run
ASAN_OPTIONS=detect_leaks=0:halt_on_error=1
Exit 0; no invalid-access diagnostic.
Full sanitized stdout retained in text_asan.output.txt.
```

6) Program sources

Each file is compiled separately. Diagnostic variants intentionally demonstrate a compiler error or invalid ownership; the corrected programs are identified in the answer and README.

text.cpp

```
#include <chrono>
#include <cstring>
#include <iostream>
#include <memory>
#include <string>
class Text {
   std::size_t n;
   char* data;
   explicit Text(std::size_t size) : n(size),data(new char[n+1]{}) {}
public:
   static int copies;
   Text(const char* s="") : n(std::strlen(s)),data(new char[n+1]) {
      std::memcpy(data,s,n+1);
   }
   Text(const Text& t) : n(t.n),data(new char[n+1]) {
      std::memcpy(data,t.data,n+1); ++copies;
   }
   Text& operator=(const Text& t) {
      if(this==&t) return *this;
      std::unique_ptr<char[]> fresh(new char[t.n+1]);
      std::memcpy(fresh.get(),t.data,t.n+1);
      delete[] data; data=fresh.release(); n=t.n; return *this;
   }
   ~Text() { delete[] data; }
   std::size_t size() const { return n; }
   friend Text operator+(const Text& a,const Text& b) {
      Text result(a.n+b.n);
      std::memcpy(result.data,a.data,a.n);
```

```
        std::memcpy(result.data+a.n,b.data,b.n+1);
        return result;
    }
    friend bool operator==(const Text& a,const Text& b) {
        return a.n==b.n && std::memcmp(a.data,b.data,a.n)==0;
    }
    friend bool operator!=(const Text& a,const Text& b) { return !(a==b); }
    friend std::ostream& operator<<(std::ostream& out,const Text& t) { return out<<t.data; }
};
int Text::copies=0;
template<class F>void timed(const char* name,F f) {
    auto start=std::chrono::steady_clock::now(); auto sum=f();
    std::cout<<name<<": "<<std::chrono::duration<double,std::milli>(
        std::chrono::steady_clock::now()-start).count()<<" ms checksum "<<sum<<'\n';
}
int main() {
    Text a("Hello"),b(" "),c("SMS"); Text::copies=0;
    Text joined=a+b+c;
    std::cout<<joined<<" copies="<<Text::copies<<'\n';
    joined=joined;
    Text empty; Text copy(joined); copy=empty;
    std::cout<<std::boolalpha<<"Self assignment "<<(joined==Text("Hello SMS"))
             <<" empty "<<(copy==empty)<<" different "<<(joined!=empty)<<'\n';
    std::string longInput(100000,'x'); Text longText(longInput.c_str());
    Text longCopy=longText; longText=empty;
    std::cout<<"Long independent copy size "<<longCopy.size()<<'\n';
    volatile int seed=7;
    timed("Text 100000 concatenations",[&] {
        std::size_t sum=0;
        for(int i=0;i<100000;++i) {
            Text x((i+seed)%2?"alpha":"omega"), y("beta");
            Text z=x+y; sum+=z.size();
        }
```

```
    return sum;
  });
  timed("std::string 100000 concatenations",[&] {
    std::size_t sum=0;
    for(int i=0;i<100000;++i) {
      std::string x=(i+seed)%2?"alpha":"omega",y="beta";
      std::string z=x+y; sum+=z.size();
    }
    return sum;
  });
}
```

7) Complex Numbers for AC Circuit Impedance

Stage 1 Requirements and data

The class stores real and imaginary parts as two doubles. A default constructor represents zero, a one real argument constructor represents a resistor or scalar, and a two argument constructor represents a general impedance. Required operations are +, -, *, /, ==, output and magnitude. Measurable requirements are: series and parallel results agree with hand calculation and std::complex<double> within an absolute error of 1e-9 for the chosen examples; a zero divisor throws domain_error; output shows rectangular form and magnitude to six decimal places. The one real argument constructor is intentionally nonexplicit so scalar values can participate in symmetric nonmember operations.

Stage 2 Class design and formulas

Complex has private re and im; public constructors, real(), imag() and magnitude(); and friend arithmetic, equality and output operators. Friends give both operands the same conversion opportunities while preserving private state. For a+jb and c+jd, addition is (a+c)+j(b+d), subtraction is (a-c)+j(b-d), and multiplication is (ac-bd)+j(ad+bc). Division is [(ac+bd)+j(bc-ad)]/(c*c+d*d). The implementation uses a scaled ratio form instead of directly squaring the denominator: when |c|>=|d|, r=d/c and den=c+d*r, giving (a+b*r)/den+j(b-a*r)/den; the other branch is analogous. This reduces some overflow hazards but is not a complete robust floating point complex division algorithm for all magnitudes. magnitude() uses hypot(re,im).

Stage 3 Circuit implementation and hand verification

Choose angular frequency omega=1000 rad/s, R=100 ohms, L=0.2 H and C=10 microfarads. Then ZR=100+j0, ZL=j*omega*L=j200, and ZC=-j/(omega*C)=-j100. Series impedance is 100+j100 with magnitude sqrt(10000+10000)=141.421356 ohms. For all three elements in parallel, admittance is 1/100+1/(j200)+1/(-j100)=0.01+j0.005 siemens. Its reciprocal is (0.01-j0.005)/(0.01 squared+0.005 squared)=80-j40 ohms, with magnitude sqrt(8000)=89.442719 ohms. The code combines the three branches pairwise using (Z1*Z2)/(Z1+Z2) and obtains the same result. The std::complex calculation independently uses the reciprocal of summed admittances. Both error checks pass and zero division is rejected.

Stage 4 Design defence and alternatives

Default construction is needed for zero and ordinary value initialization; the scalar constructor models purely real values; the pair constructor models inductive and capacitive impedances. Arithmetic

operators make series and parallel expressions readable, equality supports exact identity tests, and << provides conventional output. All operations take O(1) time and space for fixed precision doubles. Both a simple custom value and the standard type represent two double components; sizeof may include implementation specific layout details. std::complex<double> already supplies a mature mathematical interface and is preferable for general numerical work. A wrapper is still needed if a thrown exception on a zero complex denominator is part of the interface; the standard division operator does not promise this domain_error policy. Exact == does not implement numerical tolerance, so the verification uses a separate magnitude error test. Remaining limitations include near zero denominator conditioning, intermediate overflow and special infinity/NaN values. The exact zero check intentionally does not reject all small impedances, because a physical tolerance requires a domain specific scale.

Execution evidence

```
C++ COMPILER | Assignment 3 | Task07

g++ 14.2.0 | C++17 | actual captured execution

> complex
Series: 100.000000 + j100.000000 |Z|=141.421356
Parallel: 80.000000 - j40.000000 |Z|=89.442719
std::complex agreement: true true
Basic operators: 2.000000 + j2.000000 | 11.000000 + j2.000000 equality true
Rejected: zero impedance divisor

Local execution in a compiler-style presentation
```

7) Program sources

Each file is compiled separately. Diagnostic variants intentionally demonstrate a compiler error or invalid ownership; the corrected programs are identified in the answer and README.

complex.cpp

```
#include <cmath>
#include <complex>
#include <iomanip>
#include <iostream>
#include <stdexcept>
class Complex {
   double re,im;
public:
   Complex() : re(0),im(0) {}
   Complex(double r) : re(r),im(0) {}
   Complex(double r,double i) : re(r),im(i) {}
   double real() const { return re; }
   double imag() const { return im; }
   double magnitude() const { return std::hypot(re,im); }
   friend Complex operator+(const Complex&a,const Complex&b) { return {a.re+b.re,a.im+b.im}; }
   friend Complex operator-(const Complex&a,const Complex&b) { return {a.re-b.re,a.im-b.im}; }
   friend Complex operator*(const Complex&a,const Complex&b) {
      return {a.re*b.re-a.im*b.im,a.re*b.im+a.im*b.re};
   }
   friend Complex operator/(const Complex&a,const Complex&b) {
      if(b.re==0 && b.im==0) throw std::domain_error("zero impedance divisor");
      // Scaled ratio form avoids squaring the denominator components.
      if(std::abs(b.re)>=std::abs(b.im)) {
         double r=b.im/b.re,den=b.re+b.im*r;
         return {(a.re+a.im*r)/den,(a.im-a.re*r)/den};
      }
      double r=b.re/b.im,den=b.im+b.re*r;
      return {(a.re*r+a.im)/den,(a.im*r-a.re)/den};
```

```
    }
    friend bool operator==(const Complex&a,const Complex&b) { return a.re==b.re && a.im==b.im; }
    friend std::ostream& operator<<(std::ostream&out,const Complex&z) {
        return out<<z.re<<(z.im<0?" - j":" + j")<<std::abs(z.im);
    }
};
bool close(const Complex&a,const std::complex<double>&b) {
    return std::hypot(a.real()-b.real(),a.imag()-b.imag())<1e-9;
}
int main() {
    // omega=1000 rad/s, R=100 ohm, L=0.2 H, C=10 microfarad.
    const double omega=1000,R=100,L=0.2,C=10e-6;
    Complex resistor(R),inductor(0,omega*L),capacitor(0,-1/(omega*C));
    Complex series=resistor+inductor+capacitor;
    Complex parallel=(resistor*inductor)/(resistor+inductor);
    parallel=(parallel*capacitor)/(parallel+capacitor);
    std::cout<<std::fixed<<std::setprecision(6);
    std::cout<<"Series: "<<series<<" |Z|="<<series.magnitude()<<'\n';
    std::cout<<"Parallel: "<<parallel<<" |Z|="<<parallel.magnitude()<<'\n';
    std::complex<double> zr(R,0),zl(0,omega*L),zc(0,-1/(omega*C));
    auto expected=1.0/(1.0/zr+1.0/zl+1.0/zc);
    std::cout<<std::boolalpha<<"std::complex agreement: "<<close(series,zr+zl+zc)
            <<' '<<close(parallel,expected)<<'\n';
    std::cout<<"Basic operators: "<<Complex(3,4)-Complex(1,2)<<" | "
            <<Complex(1,2)*Complex(3,-4)<<" equality "<<(Complex()==Complex(0))<<'\n';
    try { auto invalid=resistor/Complex(); std::cout<<invalid; }
    catch(const std::domain_error&e) { std::cout<<"Rejected: "<<e.what()<<'\n'; }
    return !(close(series,zr+zl+zc) && close(parallel,expected));
}
```